Practice Exercise 14.1: Adaptive Agent with Feedback Memory

This practice exercise helps you apply Week 14 concepts by building a small **adaptive agent** that changes its response style based on **user feedback**. The focus is on implementing a simple feedback loop (store feedback, update behaviour, test across multiple interactions) and observing how adaptation affects the user experience.

- Building a basic agent class (Python)
- Storing feedback in memory
- Adapting response style based on feedback signals
- Testing behaviour across multiple query-feedback cycles
- (Optional) Persisting memory across sessions using a JSON file

Instructions

You will implement an adaptive agent that adjusts its style based on feedback (for example: *concise*, *detailed*, *casual*, *formal*). As you work, keep a simple log of:

- Your user query
- The agent's response style used for that turn
- The feedback you provided (e.g., "too short", "too long")
- What changed in the next response (and whether it was an improvement)

Task: Adaptive Agent with Feedback Memory

1. **Initialise the OpenAI client** and define a basic agent class (similar to the demo shown in the live session).
2. **Implement a feedback mechanism** so that the agent stores all feedback in a memory list.
3. **Modify the agent's style dynamically** based on feedback values:
    - *'too short'*
    - *'too long'*
    - *'too formal'*
4. **Add an additional feedback option** *('too casual')* that switches the style to *'formal'*.
5. **Run multiple query-feedback cycles** to test whether the agent adapts correctly as feedback changes over time.
6. **Print the agent's feedback memory** and the **final style** after several iterations.
7. **(Optional)** Extend the logic to persist memory across sessions using a JSON file.

Expected Output

By the end of this practice, you should have:

- A working script or notebook that runs multiple query-feedback cycles end-to-end.
- A feedback memory list that records the feedback you provided across turns.
- Visible evidence that the agent's response style changes based on feedback.
- A printed summary showing the agent's stored feedback and final style.
- (Optional) A JSON file that saves and reloads feedback memory across runs.

How to Approach the Project

Treat this like a small behaviour experiment, not a "perfect agent" build. Use a repeatable loop:

1. **Start simple:** Implement the agent class and one default style, and confirm that you receive a response.
2. **Add memory next:** Store each feedback item in a list and print it after every cycle to confirm it's updating.
3. **Implement one rule at a time:** Map one feedback value to one style change (then test). Avoid adding all rules at once.
4. **Force test coverage:** Run at least 5–6 cycles and intentionally alternate feedback so you can see style switching (e.g., "too short" → "too long" → "too casual" → "too formal").
5. **Check behaviour, not just code:** After each cycle, ask: Did the agent's next response actually reflect the intended style change?
6. **(Optional) Persist memory only after the core works:** JSON persistence is easiest once your in-memory loop is stable.

Minimum completion standard (recommended): Implement Tasks 1–6 and demonstrate at least three different style changes across multiple cycles.

Practice Guidelines:

This week, you will use **Vocareum** to practice building an **adaptive agent** that uses the **OpenAI API** and a simple **feedback memory** loop. You will implement a short interaction sequence (multiple turns) where the agent updates its response style based on feedback, such as *too short*, *too long*, *too formal*, and *too casual*. No submission is required for this activity; the focus is on fluency with feedback loops and adaptive behaviour in code.

To Use Chat Support for This Assignment (inside Vocareum):

- Open your practice notebook in **Vocareum**. Use the left sidebar to open the **Assistant** panel.
- Ask for clarifications, debugging help, or code snippets directly in the **Assistant** chat while you implement the feedback memory and style-switching logic.
- Monitor your **GenAI Allowance/Budget, as shown in the Assistant panel,** and plan prompts accordingly.
- If you exhaust your allowance for a task, please contact the Emeritus support team.
- **API key access:** Your course-specific API key and usage notes are provided via the guide and Vocareum instructions. Follow those steps to locate your key when needed.
- You can find detailed instructions here: Vocareum_GenAI_API_Students_Guide.pdf

Security warning: Keep your API key *private*. Do not paste it in screenshots, chats, notebooks you share, or public repos. Prefer environment variables or provided key-management cells; never hard-code keys in source files. If you suspect exposure, rotate the key immediately per the guide.

To access Vocareum:

- Go to the *Getting Started* module on your Course homepage (just below Programme Orientation).
- Click the Vocareum link (labelled *Vocareum: Professional Certificate Programme in Agentic AI and Applications*).
- Alternatively, you can also access it from the Course menu on the left side of your screen.

Note: *Working through problems independently is key to building confidence and developing a lasting understanding. This is a self-study activity for your practice and does not count towards programme completion. No submission is required. However, we highly recommend completing the activity on Vocareum to enhance your overall learning experience.*